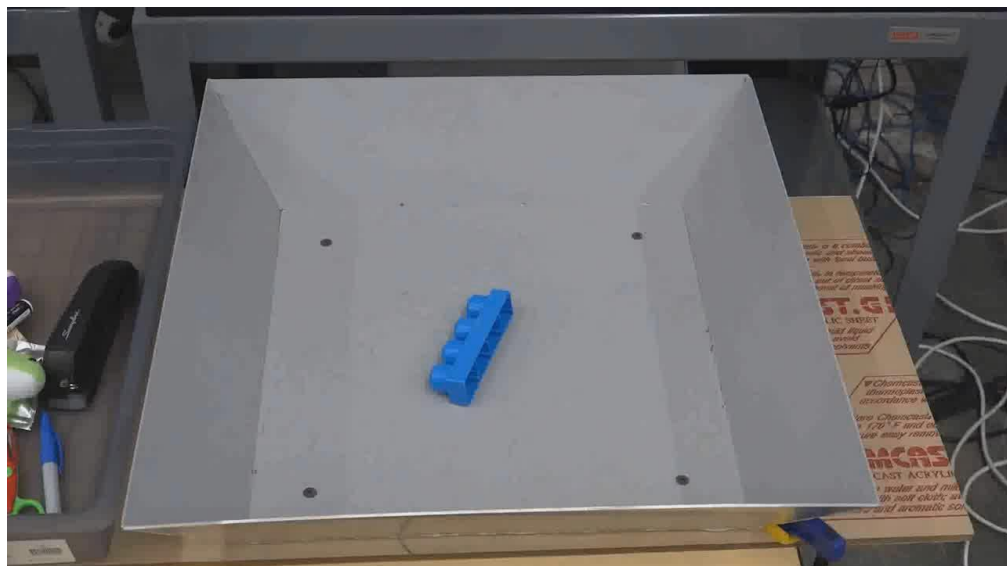
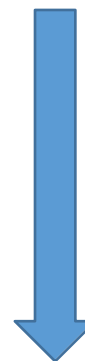
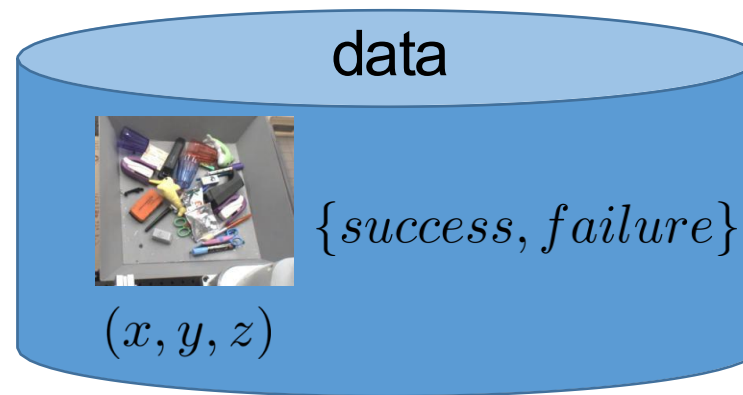
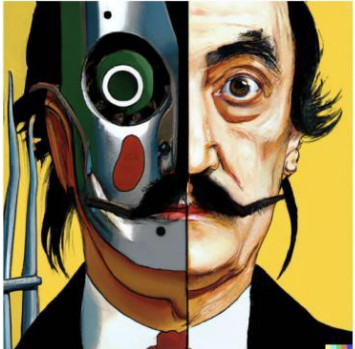


Reinforcement Learning and Self-Driving





What are some recent advances we've seen in AI?



vibrant portrait painting of Salvador Dalí with a robotic half face



a shiba inu wearing a beret and black turtleneck



a close up of a handpalm with leaves growing from it



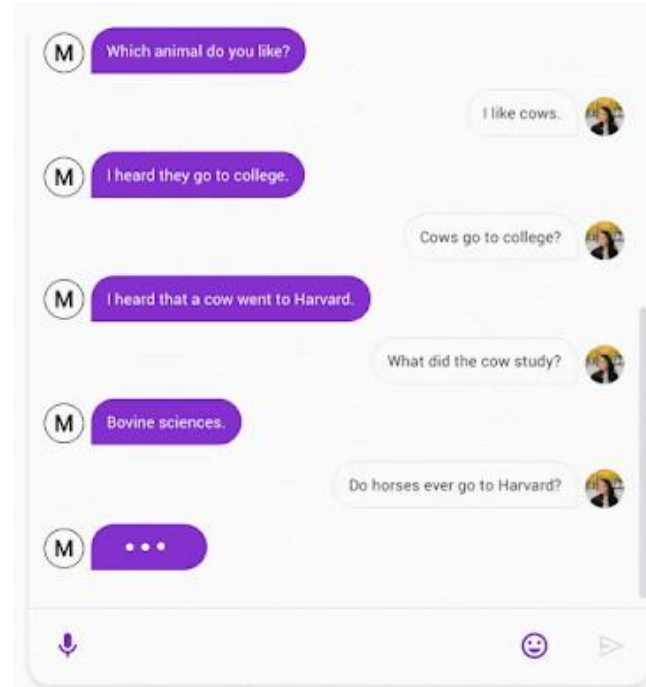
an espresso machine that makes coffee from human souls, artstation



panda mad scientist mixing sparkling chemicals, artstation



a corgi's head depicted as an explosion of a nebula



Explaining a joke

Prompt

Explain this joke:

Joke: Did you see that Google just hired an eloquent whale for their TPU team? It showed them how to communicate between two different pods!

Model Response

Prediction: TPUs are a type of computer chip that Google uses for deep learning. A "pod" is a group of TPUs. A "pod" is also a group of whales. The joke is that the whale is able to communicate between two groups of whales, but the speaker is pretending that the whale is able to communicate between two groups of TPUs.

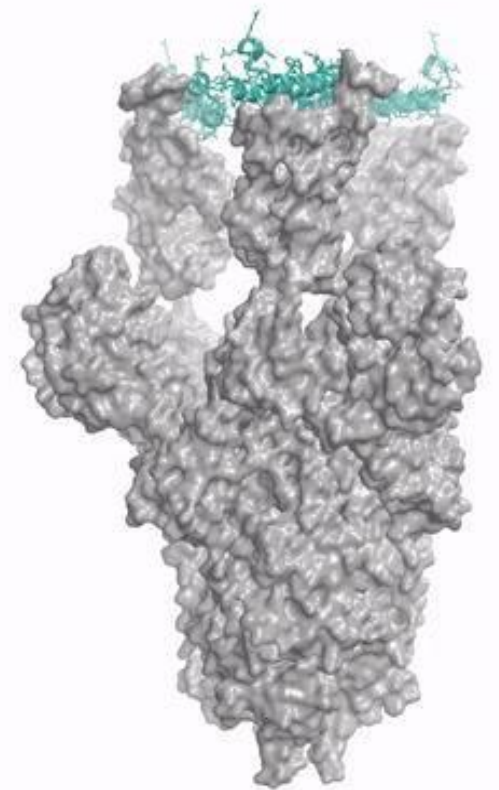
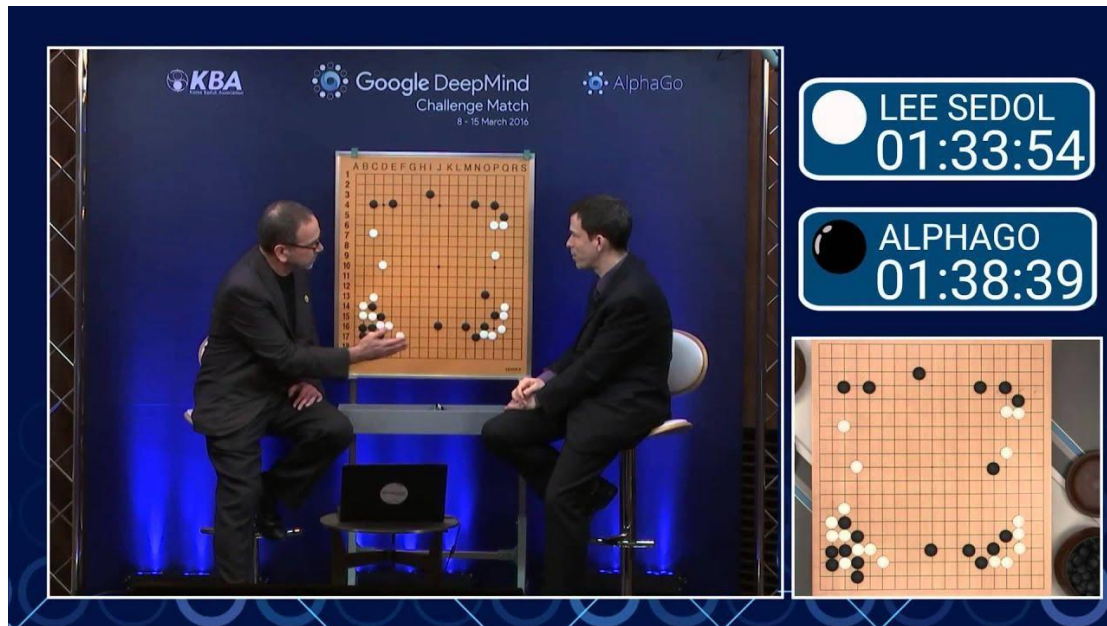


image credit: UW IPD

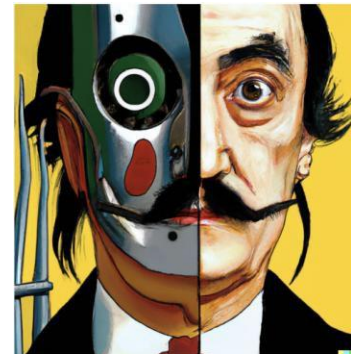
Reinforcement learning can discover new solutions

Impressive because no person had thought of it!

Impressive because it looks like something a person might draw!



“Move 37” in Lee Sedol AlphaGo match: reinforcement learning “discovers” a move that surprises everyone



vibrant portrait painting of Salvador Dalí with a robotic half face



a shiba inu wearing a beret and black turtleneck



a close up of a handpalm with leaves growing from it



an espresso machine that makes coffee from human souls, artstation



panda mad scientist mixing sparkling chemicals, artstation



a corgi's head depicted as an explosion of a nebula

What is reinforcement learning?

Mathematical formalism for learning-based decision making

Approach for learning decision making and control from experience

How is this different from other machine learning topics?

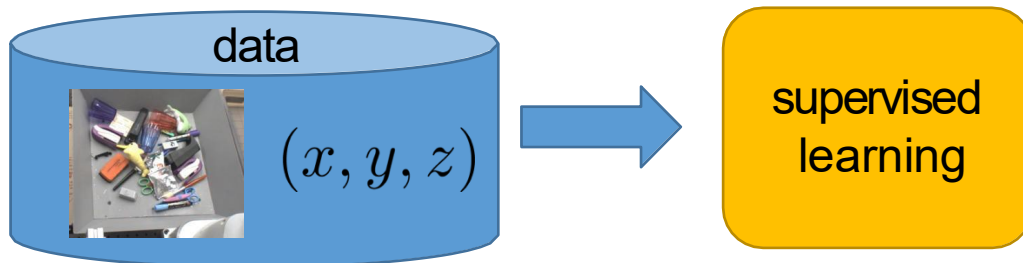
Standard (supervised) machine learning:

given $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$

learn to predict y from $\mathbf{x}$ $f(\mathbf{x}) \approx y$

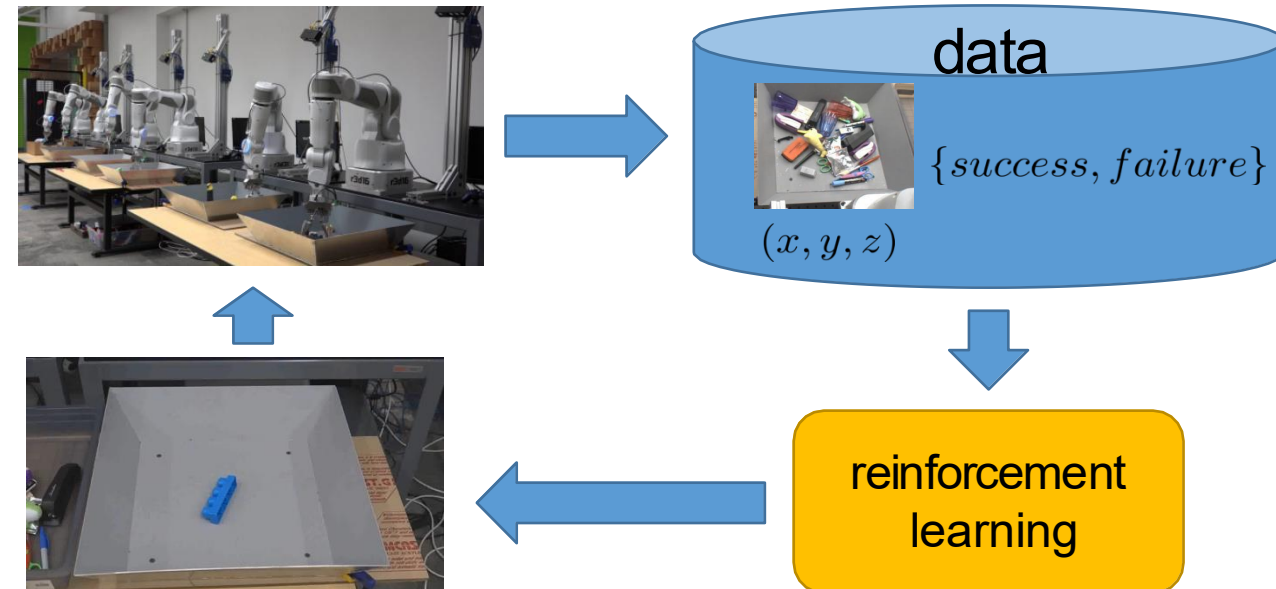
Usually assumes:

- i.i.d. data
- known ground truth outputs in training



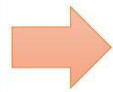
Reinforcement learning:

- Data is **not** i.i.d.: previous outputs influence future inputs!
- Ground truth answer is not known, only know if we succeeded or failed
 - more generally, we know the reward

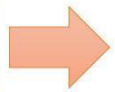


What is reinforcement learning?

supervised learning



computer
program



[object label]

input: $\mathbf{x}$

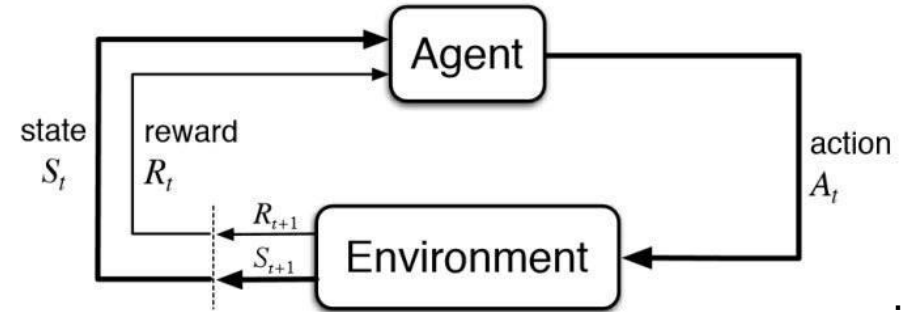
output: $\mathbf{y}$

data: $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$

goal: $f_{\theta}(\mathbf{x}_i) \approx \mathbf{y}_i$

someone gives
this to you

reinforcement learning



input: $\mathbf{s}_t$ at each time step

output: $\mathbf{a}_t$ at each time step

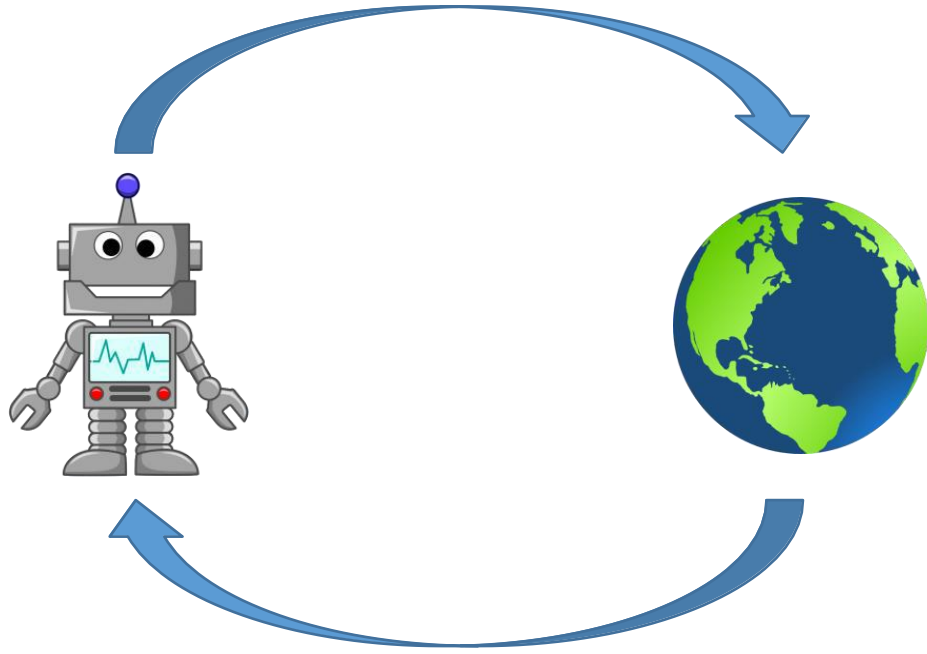
data: $(\mathbf{s}_1, \mathbf{a}_1, r_1, \dots, \mathbf{s}_T, \mathbf{a}_T, r_T)$

goal: learn $\pi_{\theta} : \mathbf{s}_t \rightarrow \mathbf{a}_t$

to maximize $\sum_t r_t$

pick your
own actions

decisions (actions)



consequences
observations (states)
rewards



Actions: muscle contractions
Observations: sight, smell
Rewards: food



Actions: motor current or torque
Observations: camera images
Rewards: task success measure (e.g., running speed)

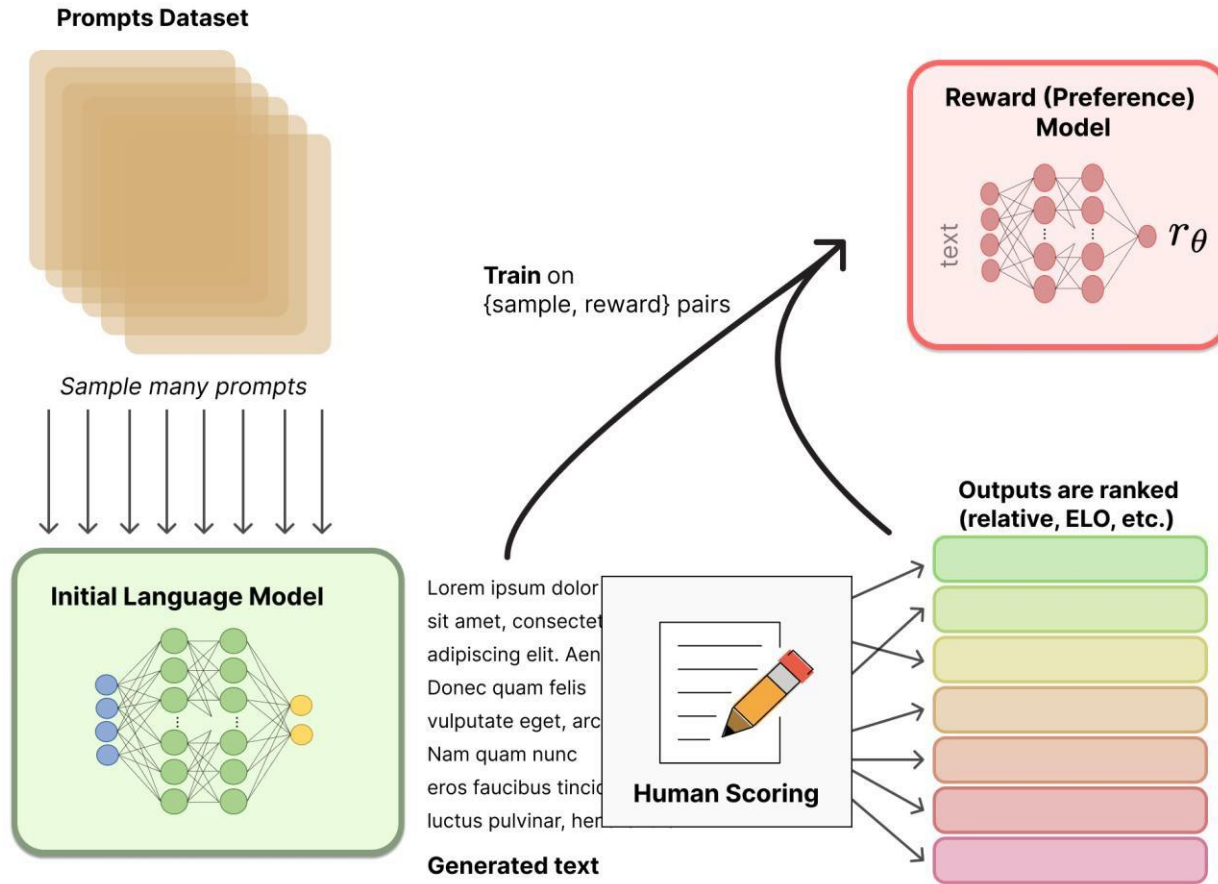


Actions: what to purchase
Observations: inventory levels
Rewards: profit

Really really complex physical tasks!



Reinforcement learning with language models



LE

Can you explain to me how RL with human feedback works for language models like I am five?



Sure! So, imagine you have a robot named Rufus who wants to learn how to talk like a human. Rufus has a language model that helps him understand words and sentences.

First, Rufus will say something using his language model. For example, he might say "I am a robot."

Then, a human will listen to what Rufus said and give him feedback on whether it sounded like a natural sentence a human would say. The human might say, "That's not quite right, Rufus. Humans don't usually say 'I am a robot.' They might say 'I'm a robot' or 'I am a machine.'"

Rufus will take this feedback and use it to update his language model. He will try to say the sentence again, using the new information he received from the human. This time, he might say "I'm a robot."

The human will listen again and give Rufus more feedback. This process will continue until Rufus can say sentences that sound natural to a human.

Over time, Rufus will learn how to talk like a human thanks to the feedback he receives from humans. This is how language models can be improved using RL with human feedback.

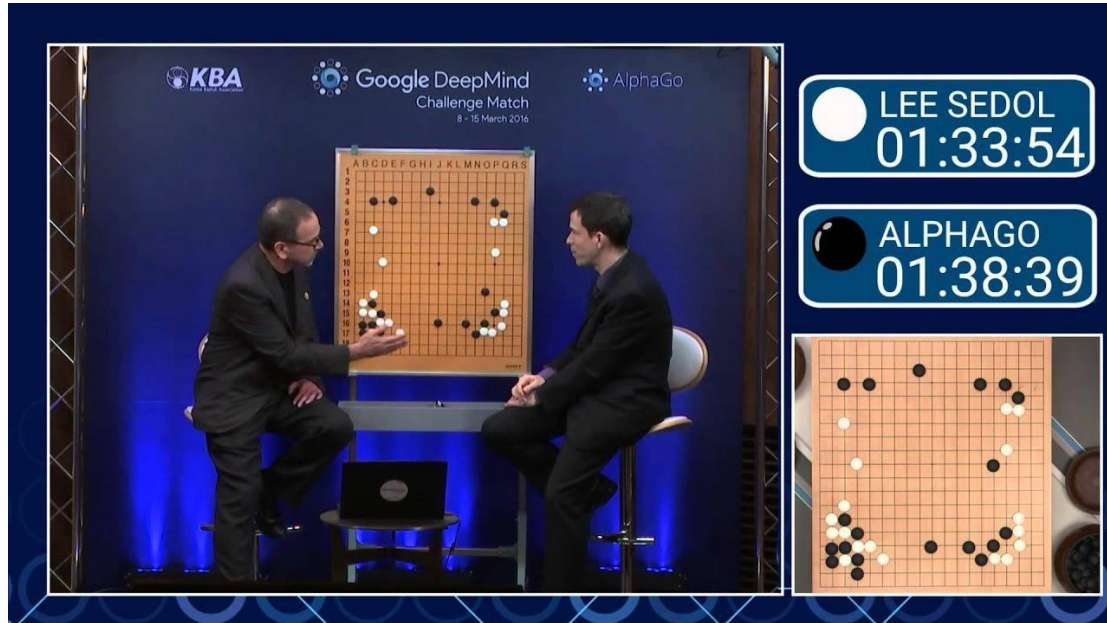
Part 3:

Why learn about reinforcement learning?

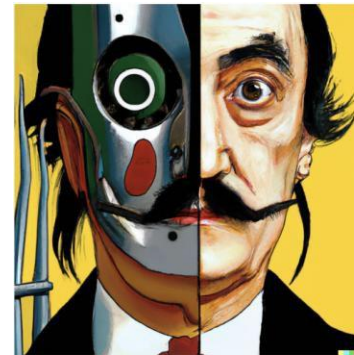
Reinforcement learning can discover new solutions

Impressive because no person had thought of it!

Impressive because it looks like something a person might draw!



“Move 37” in Lee Sedol AlphaGo match: reinforcement learning “discovers” a move that surprises everyone



vibrant portrait painting of Salvador Dalí with a robotic half face



a shiba inu wearing a beret and black turtleneck



a close up of a handpalm with leaves growing from it



an espresso machine that makes coffee from human souls, artstation



panda mad scientist mixing sparkling chemicals, artstation



a corgi's head depicted as an explosion of a nebula

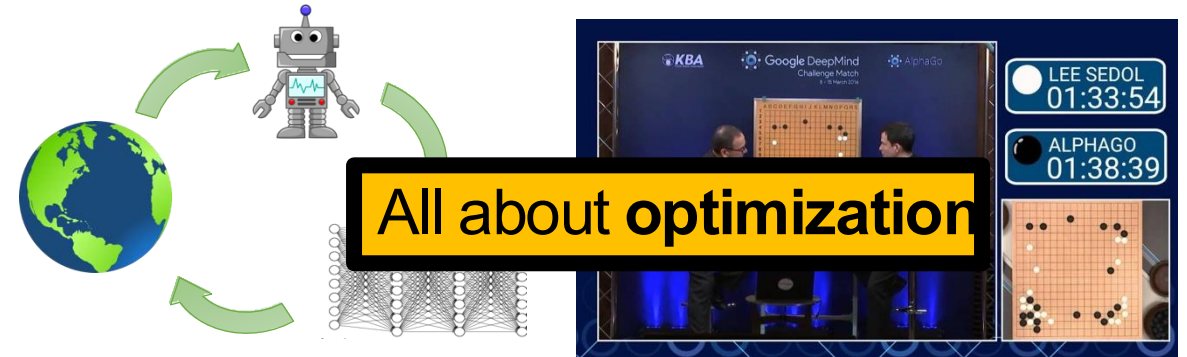
So where does that leave us?

Data-Driven AI



- + learns about the real world from data
- doesn't try to do **better** than the data

Reinforcement Learning



- + optimizes a goal with emergent behavior
- but need to figure out how to use at scale!

Data without optimization doesn't allow us to solve new problems in new ways

A bitter but misunderstood lesson



Richard Sutton

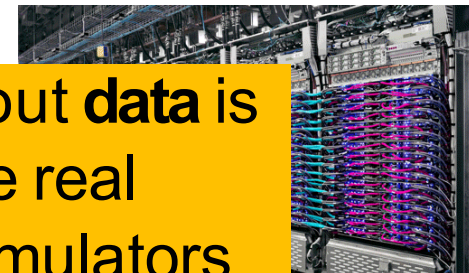
The two methods that seem to scale arbitrarily ... are *learning* and *search*.

Data without optimization doesn't allow us to solve new problems in new ways

Optimization without data is hard to apply to the real world outside of simulators

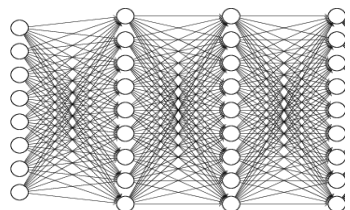


xkcd



Learning

use **data** to extract **patterns**



allows us to **understand** the world

Search

use **computation** to extract **inferences**

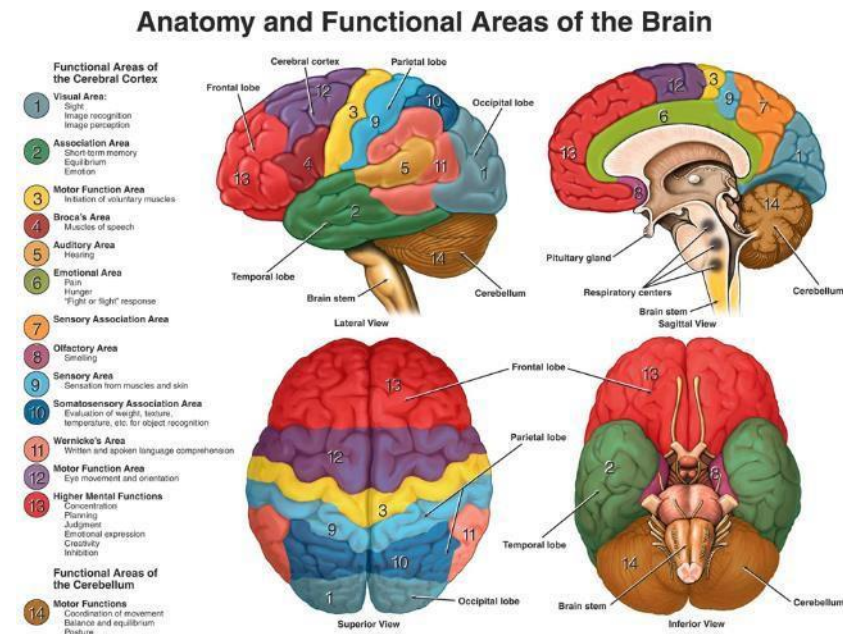
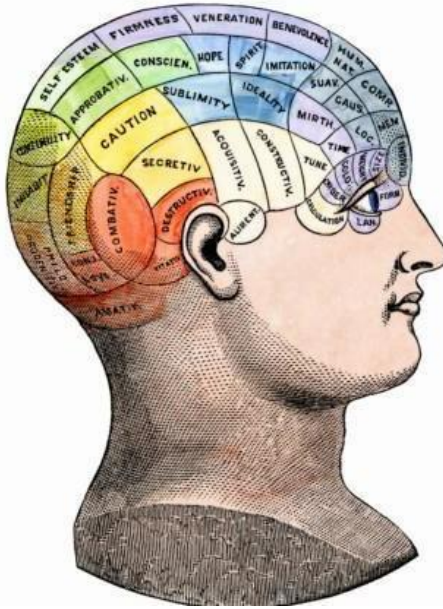
optimization

some optimization process that uses (typically iterative) computation to make rational decisions

leverages that **understanding** for **emergence**

How do we build intelligent machines?

- Imagine you have to build an intelligent machine, where do you start?

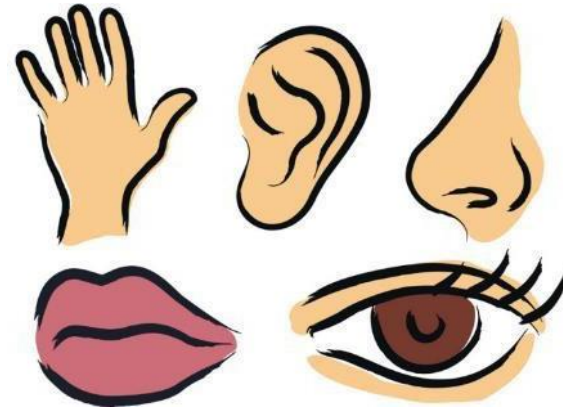


Learning as the basis of intelligence

- Some things we can all do (e.g. walking)
- Some things we can only learn (e.g. driving a car)
- We can learn a huge variety of things, including very difficult things
- Therefore our learning mechanism(s) are likely powerful enough to do everything we associate with intelligence
 - But it may still be very convenient to “hard-code” a few really important bits

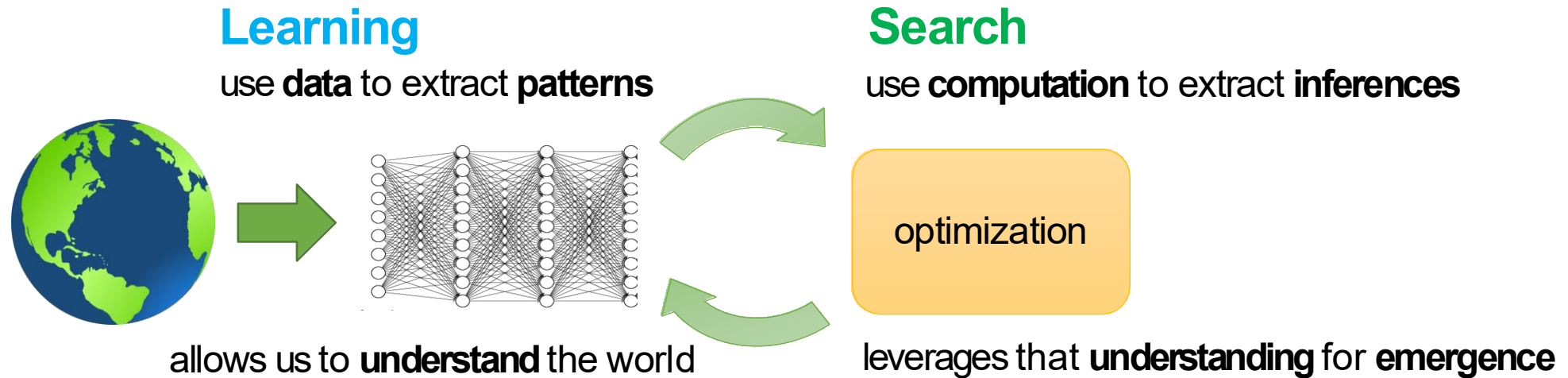
What must that single algorithm do?

- Interpret rich sensory inputs
- Choose complex actions

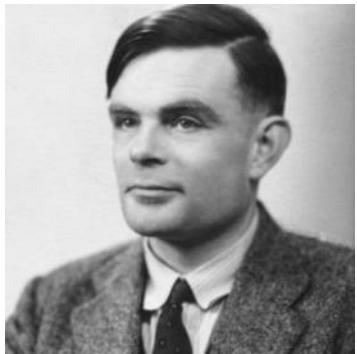


Why deep reinforcement learning?

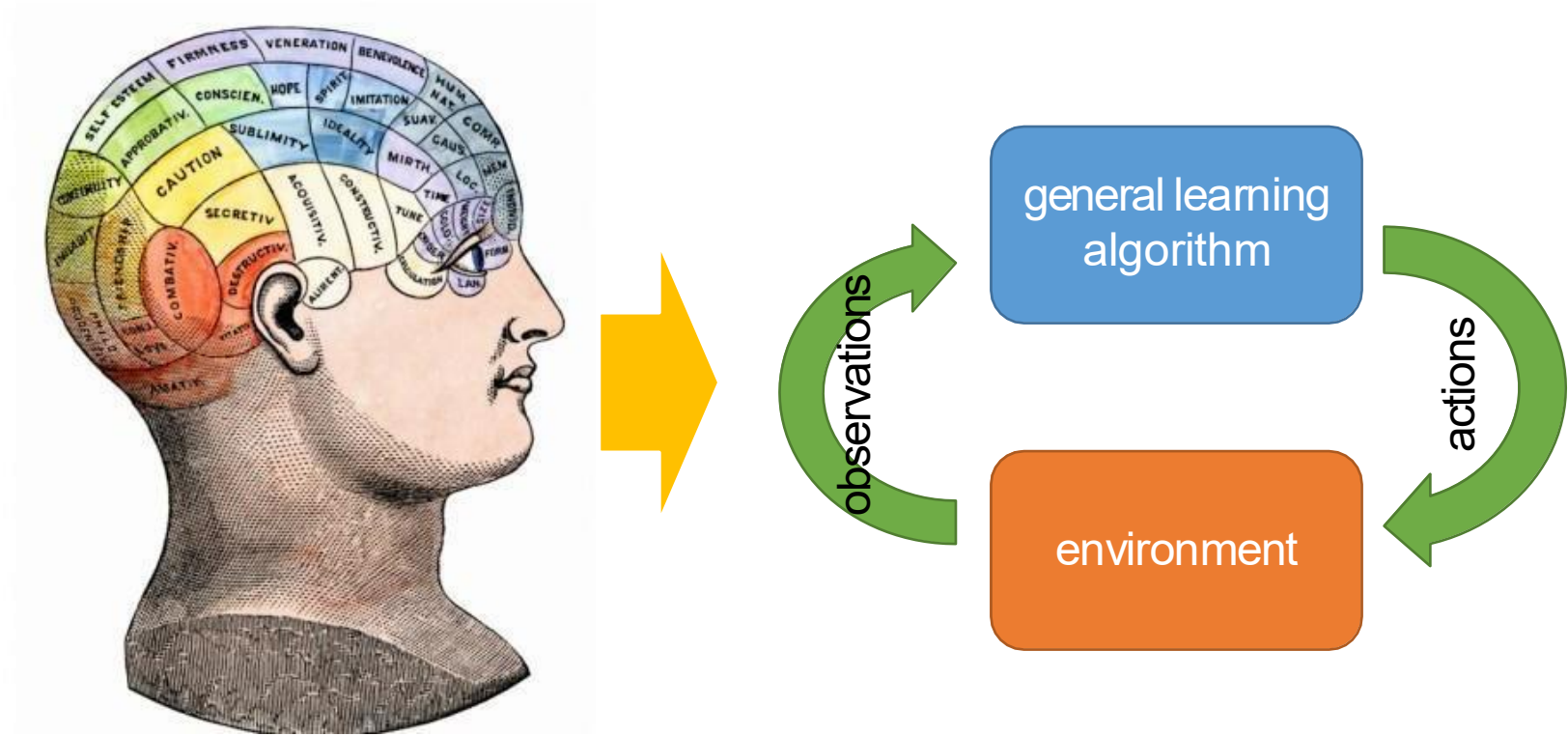
- Deep = scalable learning from large, complex datasets
- Reinforcement learning = optimization



Instead of trying to produce a program to simulate the adult mind, why not rather try to produce one which simulates the child's? If this were then subjected to an appropriate course of education one would obtain the adult brain.

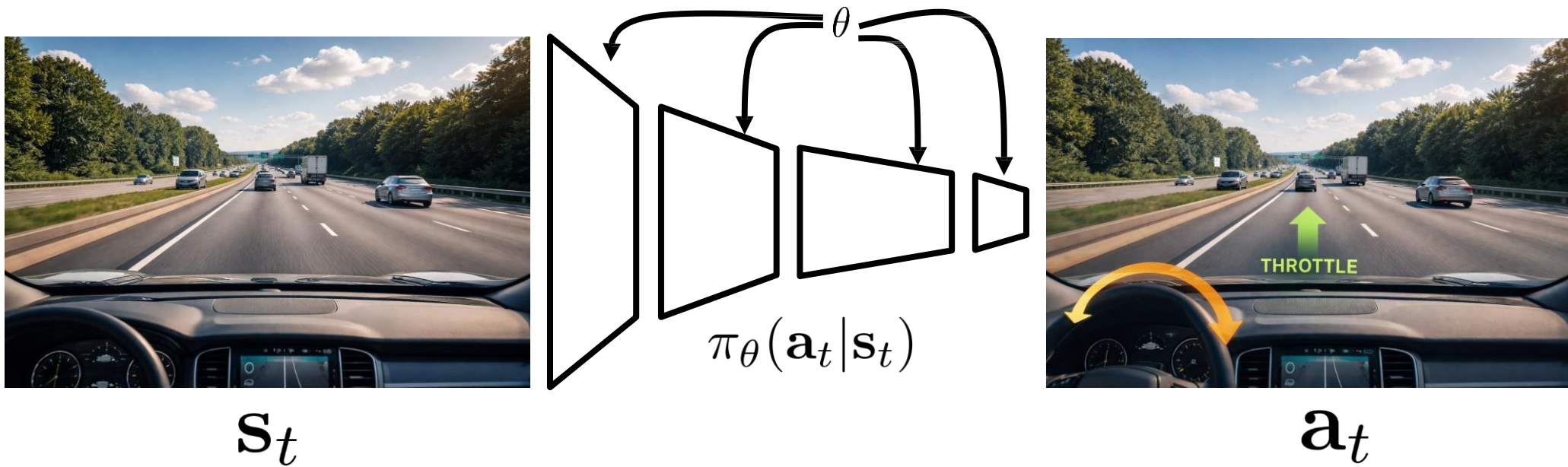


- Alan Turing



Part 1:

The Markov decision process



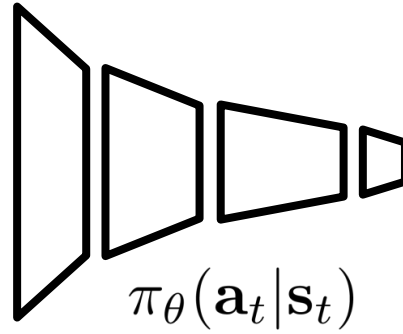
(we'll come back to partially observed settings later)

Imitation learning:
$$\arg \max_{\theta} \sum_{i=1}^N \sum_{t=1}^H \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})$$

what if we don't have good demonstration data?



$\mathbf{s}_t$



$\mathbf{a}_t$

which action is better or worse?

$r(\mathbf{s}_t, \mathbf{a}_t)$: reward function

tells us which states and actions are better

$\mathbf{s}_t$, $\mathbf{a}_t$, $r(\mathbf{s}_t, \mathbf{a}_t)$, and $p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ define
Markov decision process



high reward



low reward

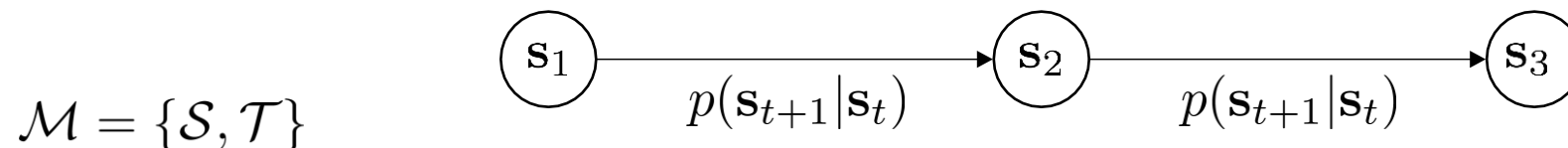
Basic definition: Markov chain

The most basic probabilistic model of a dynamical system

No actions yet, only states



Andrey Markov



Markov property

$$\mathbf{s}_{t+1} \perp \mathbf{s}_{t-1} | \mathbf{s}_t$$

$\mathcal{S}$ – state space

states $\mathbf{s} \in \mathcal{S}$ (discrete or continuous)

$\mathcal{T}$ – transition operator

$$p(\mathbf{s}_{t+1} | \mathbf{s}_t)$$

why “operator”?

$$p(\mathbf{s}_{t+1}) = \sum_{\mathbf{s}_t} p(\mathbf{s}_{t+1} | \mathbf{s}_t) p(\mathbf{s}_t)$$

current
state
marginal

$$\mu_t = \begin{bmatrix} p(\mathbf{s}_t = 1) \\ p(\mathbf{s}_t = 2) \\ \dots \end{bmatrix}$$

$$\text{let } \mathcal{T}_{i,j} = p(\mathbf{s}_{t+1} = i | \mathbf{s}_t = j) \quad \mu_{t+1} = \mathcal{T} \mu_t$$

This bit of linear algebra will be important later!

Basic definition: Markov decision process

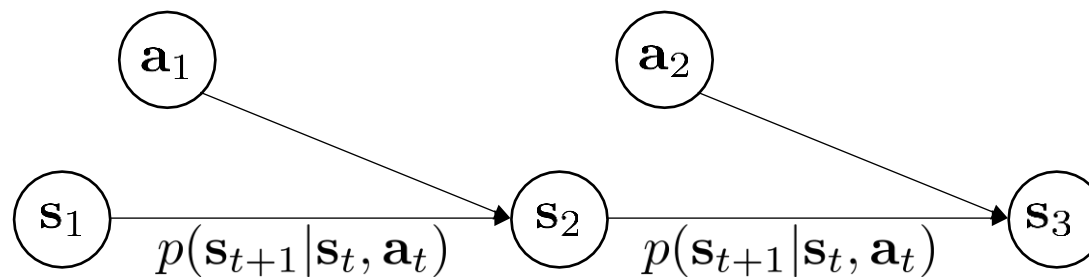
$$\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{T}, r\}$$

states $\mathbf{s} \in \mathcal{S}$

actions $\mathbf{a} \in \mathcal{A}$

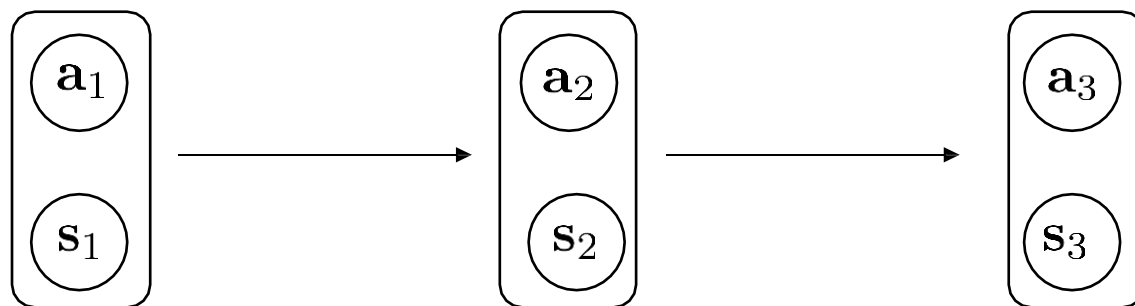
transition operator $\mathcal{T}$

reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$



Richard Bellman

Can we turn a Markov decision process into a Markov chain?



$$p((\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) | (\mathbf{s}_t, \mathbf{a}_t)) = p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) \pi_{\theta}(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})$$

Part 2: Defining the objective



The objective of RL

$r(\mathbf{s}_t, \mathbf{a}_t)$: reward function

tells us which states and actions are better



high reward



low reward

by the time you're about to hit someone, it's too late!

need to optimize for **long-term** reward

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_H, \mathbf{a}_H)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^H \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

Expectations and stochastic systems

$$\theta^* = \arg \max_{\theta} E_{(\mathbf{s}, \mathbf{a}) \sim p_{\theta}(\mathbf{s}, \mathbf{a})} [r(\mathbf{s}, \mathbf{a})]$$

infinite horizon case

$$\theta^* = \arg \max_{\theta} \sum_{t=1}^T E_{(\mathbf{s}_t, \mathbf{a}_t) \sim p_{\theta}(\mathbf{s}_t, \mathbf{a}_t)} [r(\mathbf{s}_t, \mathbf{a}_t)]$$

finite horizon case

In RL, we almost always care about *expectations*



The main idea

Bellman principle

The value of a decision equals the immediate reward plus the discounted value of what happens next.

For an action-value function:

$$Q(s, a) \approx \text{immediate reward} + \text{discounted future value.}$$

In reinforcement learning, $Q(s, a)$ answers:

$$Q(s, a) = \text{How good is it to take action } a \text{ in state } s?$$

Markov decision process setup

At time t , the agent observes a state, chooses an action, receives reward, and moves to the next state:

$$s_t \rightarrow a_t \rightarrow r_{t+1}, s_{t+1}.$$



Important quantities:

$$\gamma \in [0, 1), \quad \pi(a \mid s), \quad p(s', r \mid s, a).$$

Definition of the action-value function

For a fixed policy π , the action-value function is

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right]$$

Meaning:

- First, take action a in state s .
- Then, follow policy π afterward.
- $Q^{\pi}(s, a)$ is the expected discounted total reward.

Bellman equation for $Q^\pi(s, a)$

The Bellman equation decomposes the return into one step plus the future:

$$Q^\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') Q^\pi(s', a') \right]$$

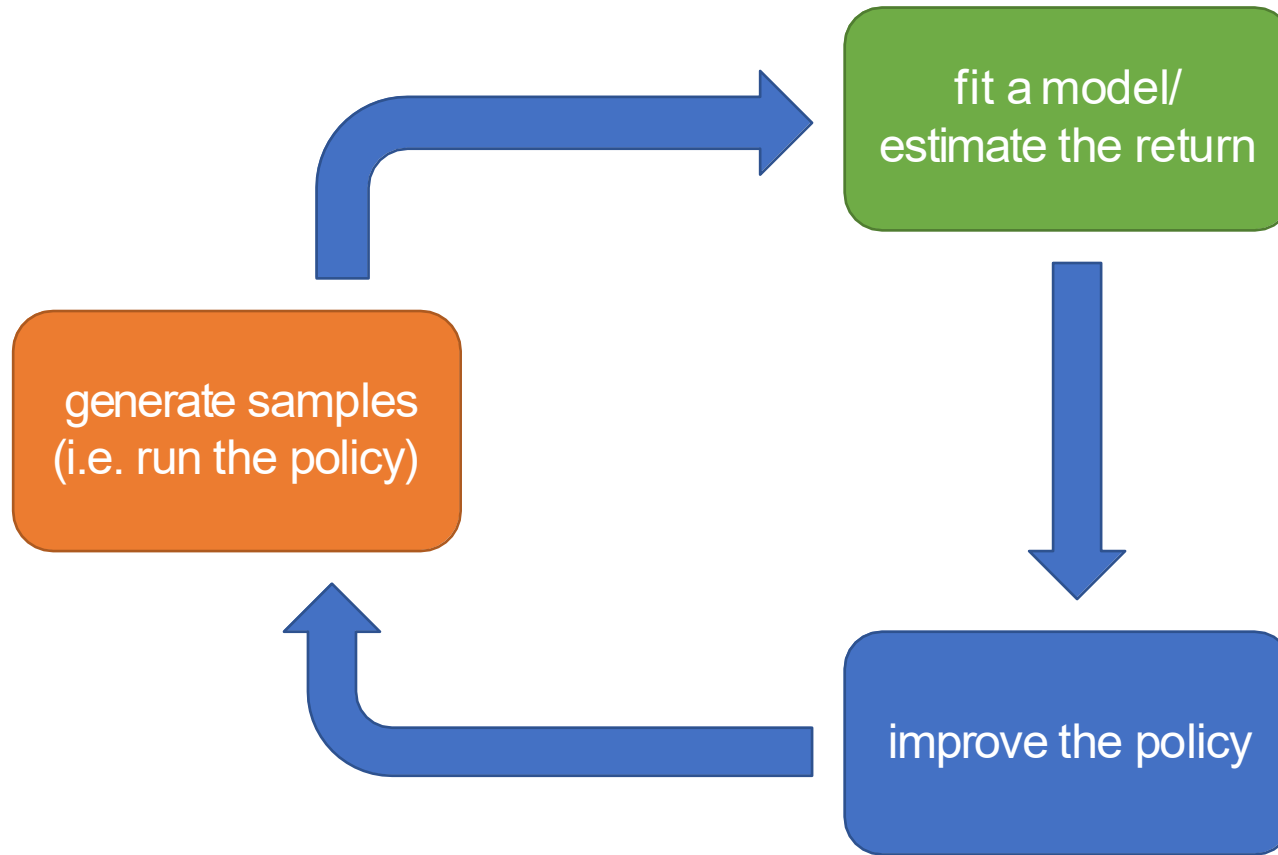
Compact expectation form:

$$Q^\pi(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \mathbb{E}_{a' \sim \pi(\cdot \mid s_{t+1})} Q^\pi(s_{t+1}, a') \mid s_t = s, a_t = a \right]$$

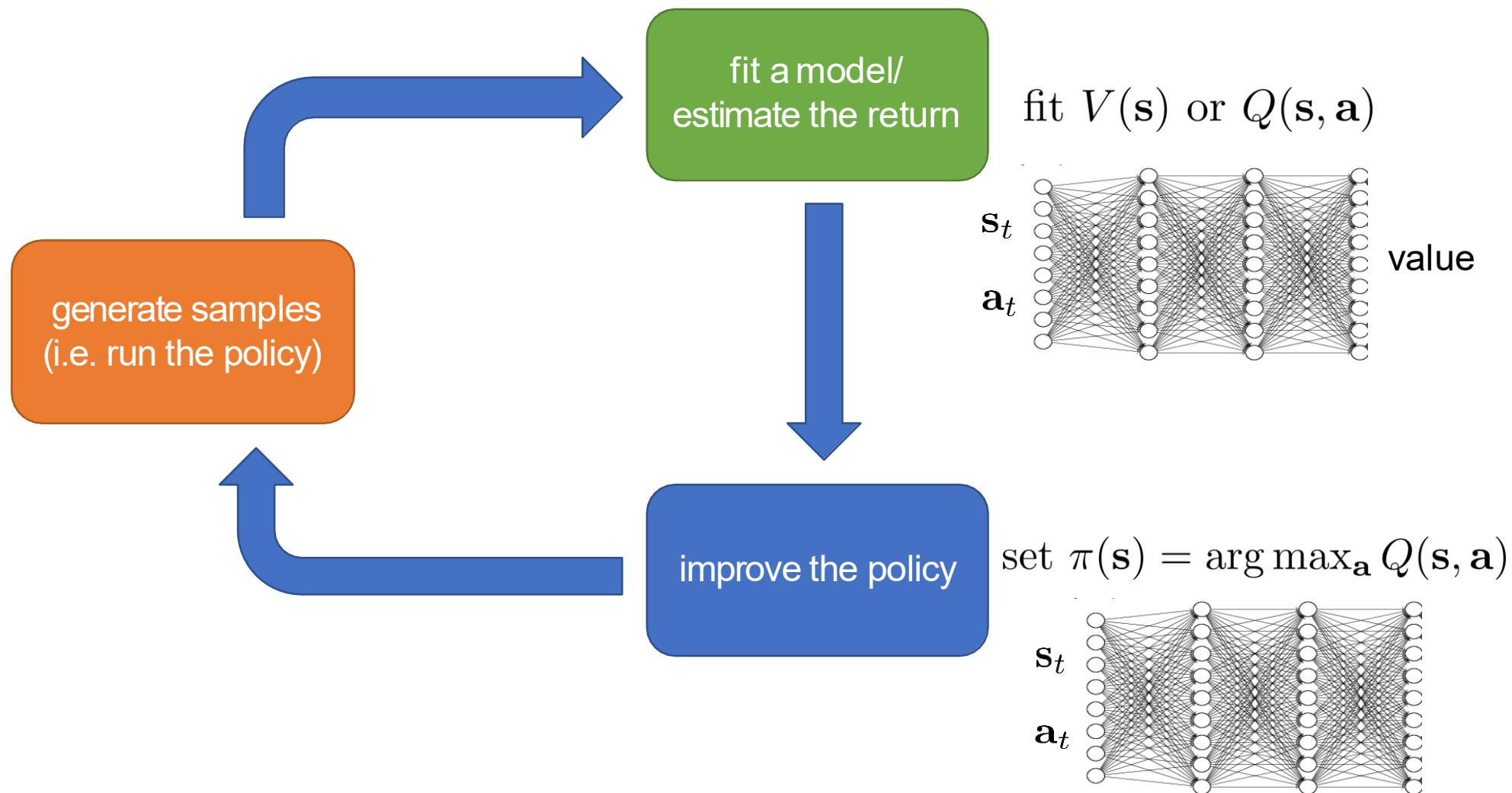
Part 3:

Anatomy of an RL algorithm

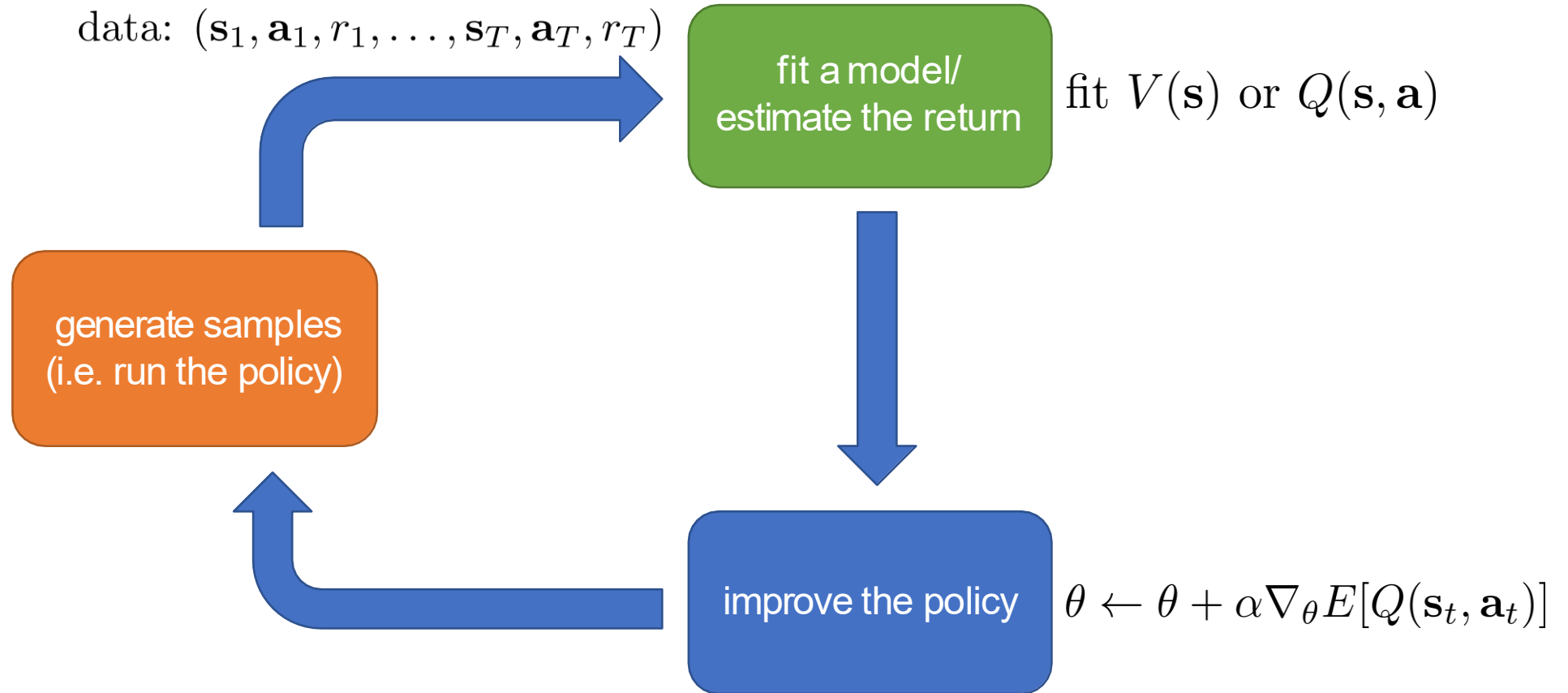
The anatomy of a reinforcement learning algorithm



Value function based algorithms



Actor-critic: value functions + policy gradients



Example 1: Atari games with Q-functions

- Playing Atari with deep reinforcement learning, Mnih et al. '13
- Q-learning with convolutional neural networks

